Práctica 2: Señales y Semáforos

Comunicación entre procesos mediante señales POSIX y sincronización con semáforos.

FECHA DE ENTREGA: 7-10 DE ABRIL

ANTES DEL COMIENZO DE LA CLASE DE PRÁCTICAS

Contenidos de la práctica

01

Señales

Introducción, envío, tratamiento, temporización y herencia padre-hijo.

02

Semáforos

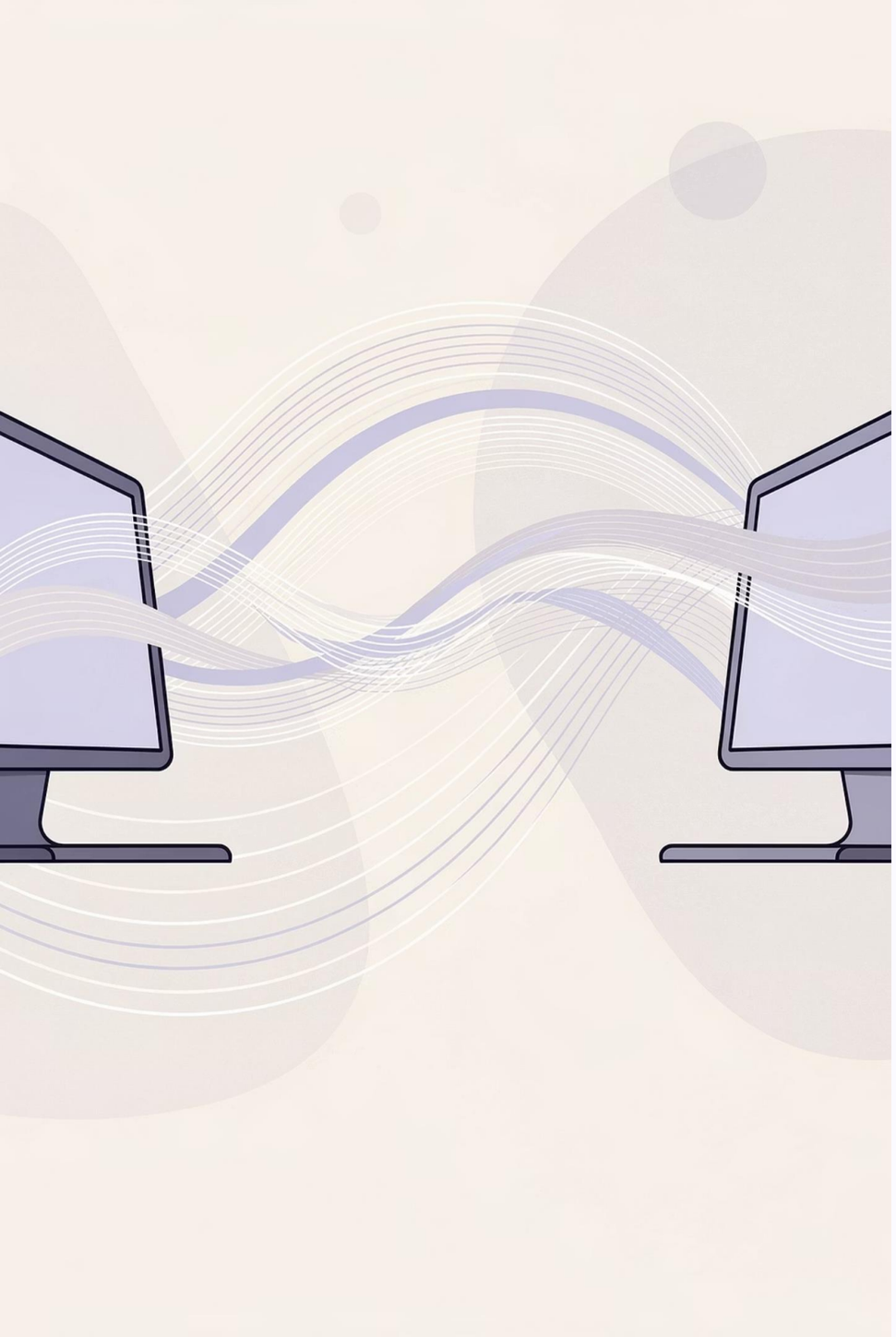
Introducción, creación, eliminación, operaciones y concurrencia.

03

Ejercicio de codificación

Sistema de votación multiproceso basado en Blockchain (10 puntos).

Los ejercicios cortos son de aprendizaje sin puntuación. Solo se entrega el ejercicio de codificación (código + documentación).



PRIMERA PARTE

Señales

Introducción a las señales

Las **señales** son una forma limitada de comunicación entre procesos. Son a los procesos lo que las interrupciones son al procesador: cuando un proceso recibe una señal, detiene su ejecución, bifurca a la rutina de tratamiento y luego continúa desde el punto de bifurcación.

Enviadas por el núcleo

- `SIGINT` al pulsar `[Ctrl]+[C]`
- `SIGSEGV` por fallo de segmentación

Enviadas entre procesos

Los procesos pueden enviarse señales entre sí mediante la función `kill`, siempre que tengan el mismo UID.

Reacciones ante una señal

Cuando un proceso recibe una señal puede reaccionar de **tres** formas:

1

Ignorar la señal

El proceso es inmune a la señal recibida.

2

Manejador por defecto

Rutina aportada por el núcleo. Generalmente termina el proceso con `exit`. Algunas señales generan además un fichero `core` con volcado de memoria.

3

Manejador propio

Rutina codificada por el programador. El núcleo la invoca al recibir la señal y el programador define las acciones pertinentes.

Tabla de señales UNIX

Cada señal tiene un número entero positivo asociado, definido en `<signal.h>`.

Señal	Valor	Acción	Descripción
SIGHUP	1	Term	Hangup en terminal controlador
SIGINT	2	Term	Interrupción desde teclado
SIGQUIT	3	Core	Quit desde teclado
SIGKILL	9	Term	Señal de terminación forzosa
SIGUSR1	10	Term	Señal definida por el usuario 1
SIGSEGV	11	Core	Referencia de memoria inválida
SIGUSR2	12	Term	Señal definida por el usuario 2
SIGALRM	14	Term	Señal de temporizador de alarm(2)
SIGTERM	15	Term	Señal de terminación
SIGCHLD	17	Ign	Hijo detenido o terminado
SIGSTOP	19	Stop	Detener proceso
SIGTSTP	20	Stop	Detención desde terminal

Estas son solo algunas de las señales.
Prueba encontrar con **man** la lista completa de señales en la entrada de **signal**

Envío de señales

Desde shell: comando `kill`

Linux dispone del comando `kill` para enviar señales a procesos. Permite listar las señales disponibles y enviarlas por número o nombre.

Ejercicio 1: Buscar en el manual cómo listar señales con `kill`. ¿Qué número tienen SIGKILL y SIGSTOP?

Desde C: función `kill`

- `pid > 0`: envía al proceso con PID `pid`
- `pid = 0`: envía al grupo del proceso llamante
- `pid = -1`: envía a todos los procesos con permiso
- `pid < -1`: envía al grupo con ID `-pid`
- `sig = 0`: comprobación de errores, sin señal

Ejercicio 2: Envío de señales (`sig_kill.c`)

a) Completar el programa en C anterior, de manera que reproduzca de forma limitada la funcionalidad del comando de shell kill con un formato similar:

$ ./ sig_kill -< signal > <pid >

El programa debe recibir dos parámetros: el primero, <signal> representa el identicador numérico de la señal a enviar; el segundo, <pid>, el PID del proceso al que se enviará la señal.

b) Probar el programa enviando la señal SIGSTOP de una terminal a otra (cuyo PID se puede averiguar fácilmente con el comando ps). ¿Qué sucede si se intenta escribir en la terminal a la que se ha enviado la señal? ¿Y después de enviarle la señal SIGCONT?

Tratamiento de señales: `sigaction`

La función `sigaction` permite **capturar señales** y modificar el comportamiento del proceso. Sus campos principales son:

`sa_handler`

Función manejadora. Puede ser `SIG_DFL` (defecto), `SIG_IGN` (ignorar) o la dirección de una rutina propia.

`sa_sigaction`

Manejador extendido con prototipo `(int, siginfo_t*, void*)`. Se usa con la bandera `SA_SIGINFO` para recibir información adicional.

`sa_mask`

Máscara de señales adicionales bloqueadas durante la ejecución del manejador. La señal capturada se bloquea por defecto.

`sa_flags`

Banderas para modificar el comportamiento del manejador.

Ejercicio 3: Captura de señales (`sig_capture.c`)

a) ¿La llamada a sigaction supone que se ejecute la función handler?

b) ¿Se bloquea alguna señal durante la ejecución de la función handler?

c) ¿Cuándo aparece el printf en pantalla?

d) Modicar el programa anterior para que no capture SIGINT. ¿Qué sucede cuando se pulsa Ctrl + C ? En general, ¿qué ocurre por defecto cuando un programa recibe una señal y no tiene instalado un manejador?

e) A partir del código anterior, escribir un programa que capture todas las señales (desde la 1 hasta la 31) usando el mismo manejador. ¿Se pueden capturar todas las señales? ¿Por qué?

Manejadores de señales: buenas prácticas

La llamada al manejador es **asíncrona**: puede ocurrir en cualquier instante. Por eso se recomienda que el manejador solo controle que se ha recibido la señal, y que la lógica asociada se ejecute en el flujo normal del programa.

Consultar el manual de `signal-safety` para conocer las funciones seguras que pueden usarse dentro de un manejador.

Ejercicio 4: Captura mejorada (`sig_capture_improved.c`)

a) En esta versión mejorada del programa del Ejercicio 3, ¿en qué líneas se realiza realmente la gestión de la señal?

b) ¿Por qué, en este caso, se permite el uso de variables globales?

Protección de zonas críticas: máscaras de señales

Las **máscaras de señales** permiten bloquear señales durante la ejecución de código crítico. **Bloquear ≠ ignorar**: la señal queda pendiente hasta que se desbloquee.

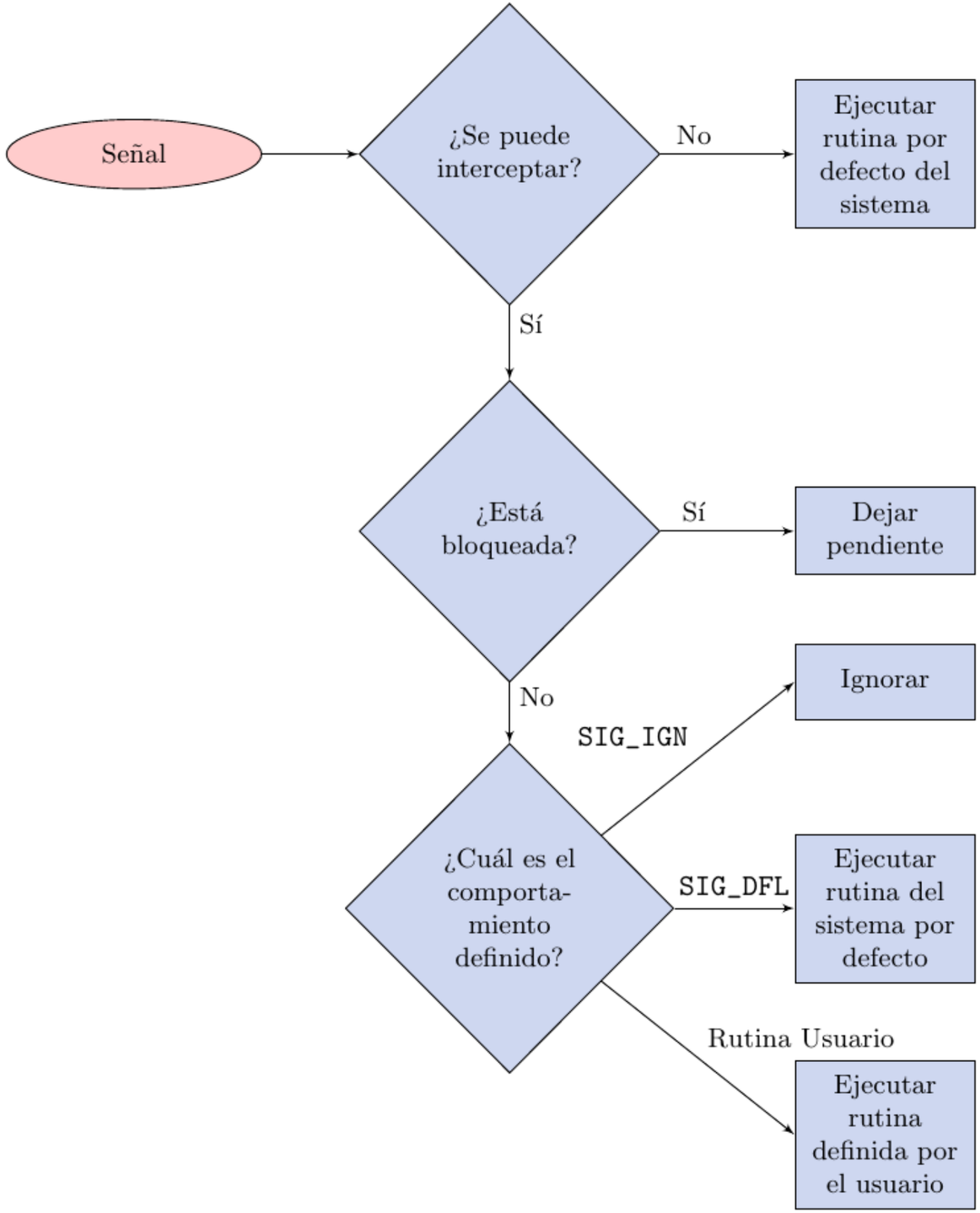
Funciones de conjunto (`sigsetops`)

- `sigemptyset`: conjunto vacío
- `sigfillset`: todas las señales
- `sigaddset` / `sigdelset`: añadir/eliminar
- `sigismember`: comprobar pertenencia

Aplicar máscara: `sigprocmask`

- `SIG_BLOCK`: unión con máscara actual
- `SIG_SETMASK`: reemplaza la máscara
- `SIG_UNBLOCK`: desbloquea señales del conjunto

En procesos multihilo usar `pthread_sigmask` en lugar de `sigprocmask`.
Para señales bloqueadas pendientes: `sigpending`.



Flujo de manejo de señales

Figura 1: Diagrama de flujo de recepción y manejo de señales por el sistema operativo.

Ejercicio 5: Bloqueo de señales (`sig_sigset.c`)

```
int main(void) {
    sigset_t set, oset;
    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);
    sigaddset(&set, SIGUSR2);
      if (sigprocmask(SIG_BLOCK, &set, &oset) < 0) {
          perror("sigprocmask"); exit(EXIT_FAILURE);
    }
    printf("Waiting signals (PID = %d)\n", getpid());
    printf("SIGUSR1 and SIGUSR2 are blocked\n");
    pause();
    printf("End of program\n");
    exit(EXIT_SUCCESS);
}
```

Aviso: versión reducida del código para las diapositivas. Consultar el enunciado

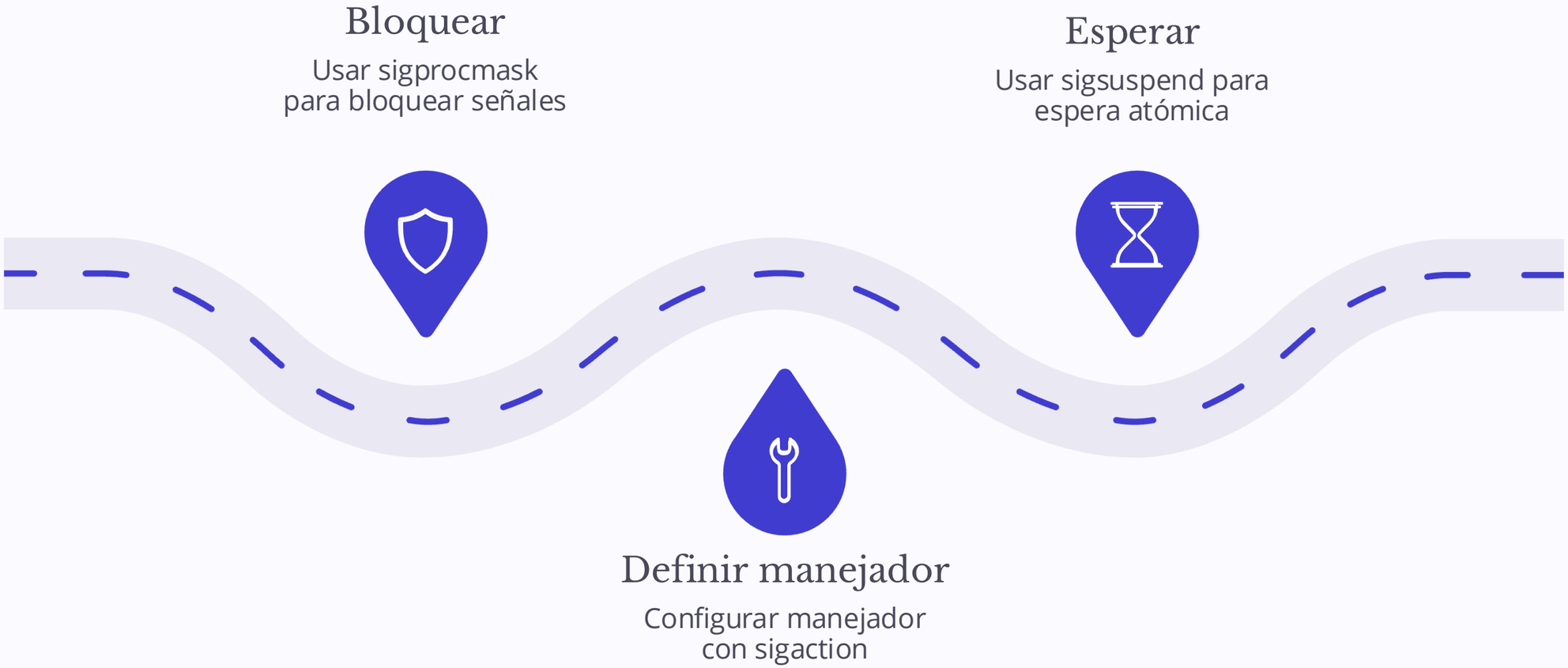
¿Qué sucede cuando el programa anterior recibe SIGUSR1/SIGUSR2?

¿Qué sucede cuando el programa anterior recibe SIGINT?

¿Y si modificamos pause por sleep?

Espera de señales: `sigsuspend`

La función `sigsuspend` es más potente que `pause`: de forma **atómica** sustituye la máscara actual por la recibida y queda en espera no activa. Permite seleccionar qué señal esperar y evita condiciones de carrera.



El uso correcto requiere bloquear las señales *antes* de `sigaction` y desbloquearlas con `sigsuspend`, garantizando que no se pierda ninguna señal entre ambas llamadas.

Servicio de temporización: `alarm`

Función `alarm`

Provoca que `SIGALRM` se envíe al propio proceso tras los segundos indicados como parámetro.

Nota: `alarm` y `sleep` pueden interferir entre sí. No mezclarlas.

API POSIX de temporizadores

Para mayor potencia: número arbitrario de temporizadores, selección de reloj, repetición periódica y visibilidad en `/proc`.

- `timer_create`
- `timer_settime` / `timer_gettime`
- `timer_getoverrun`
- `timer_delete`

Ejercicio 6: Gestión de la alarma (`sig_alarm.c`)

```
#define SECS 10

void handler_SIGALRM(int sig) {
    printf("\nThese are the numbers I had time to count\n");
    exit(EXIT_SUCCESS);
}

int main(void) {
    struct sigaction act;
    long int i;
    act.sa_handler = handler_SIGALRM;
    sigemptyset(&(act.sa_mask));
    act.sa_flags = 0;
    sigaction(SIGALRM, &act, NULL);
    if (alarm(SECS))
         fprintf(stderr, "There is a previously established alarm\n");
    fprintf(stdout, "The count starts (PID=%d)\n", getpid());
    for (i = 0; i++ < SECS) {
        fprintf(stdout, "%10ld\r", i); fflush(stdout);
    }
    fprintf(stdout, "End of program\n");
    exit(EXIT_SUCCESS);
}
```

Aviso: versión reducida del código para las diapositivas. Consultar el enunciado

a) ¿Qué sucede si, mientras se realiza la cuenta, se envía la Señal SIGALARM al proceso?

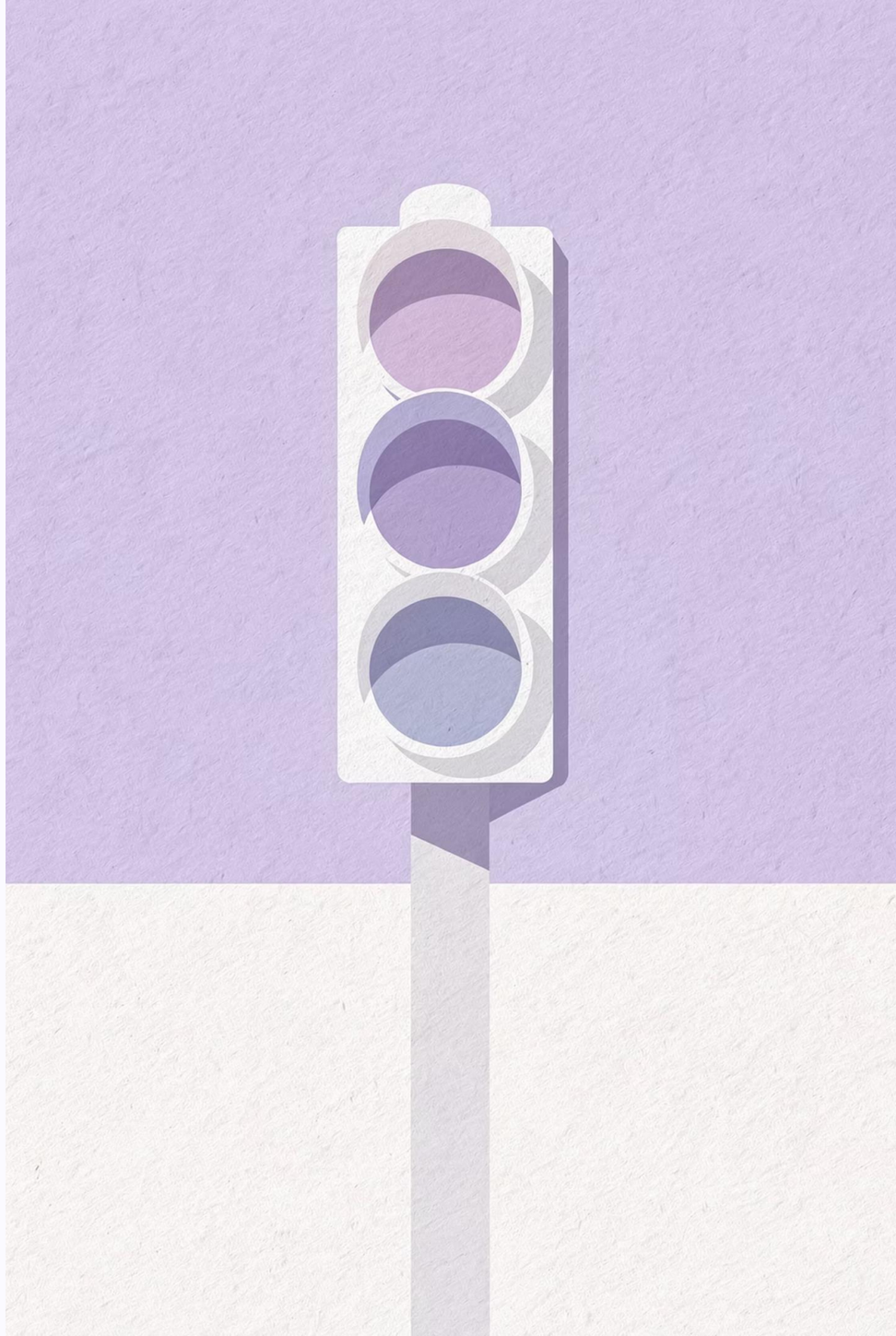
b) ¿Qué pasa si se comenta la llamada a `sigaction`?

Herencia entre procesos padre-hijo

Propiedad	**Tras** `fork`	**Tras** `exec`
Máscara de señales bloqueadas	Heredada	Heredada
Lista de señales pendientes	Vacía en el hijo	Mantenida
Rutinas de manejo	Heredadas	No heredadas
Alarmas	No heredadas	Heredadas

SEGUNDA PARTE

Semáforos



Introducción a los semáforos

Los **semáforos** son un mecanismo de sincronización del sistema operativo que palía los riesgos del acceso concurrente a recursos compartidos. Se comportan como variables enteras con operaciones atómicas.

Semáforo binario

Valores 0 y 1. Garantiza **exclusión mutua**: controla que solo un proceso escriba en memoria compartida al mismo tiempo.

Semáforo N-ario

Valores de 0 a N. Permite que un **número determinado** de procesos trabaje concurrentemente en tareas no críticas (p.ej., lectura simultánea).

Operaciones fundamentales

Down (sem_wait)

Petición del semáforo. Si el valor es mayor que 0, concede el paso y decrementa atómicamente. Si está a 0, el proceso queda bloqueado en espera no activa hasta que el valor suba.

Up (sem_post)

Liberación del semáforo. Incrementa atómicamente el valor en una unidad. Si había procesos bloqueados esperando, uno de ellos obtiene acceso.

Ambas operaciones son **atómicas**: se completan como una unidad sin posibilidad de interrupción por otros procesos.

Semáforo para ordenar ejecución

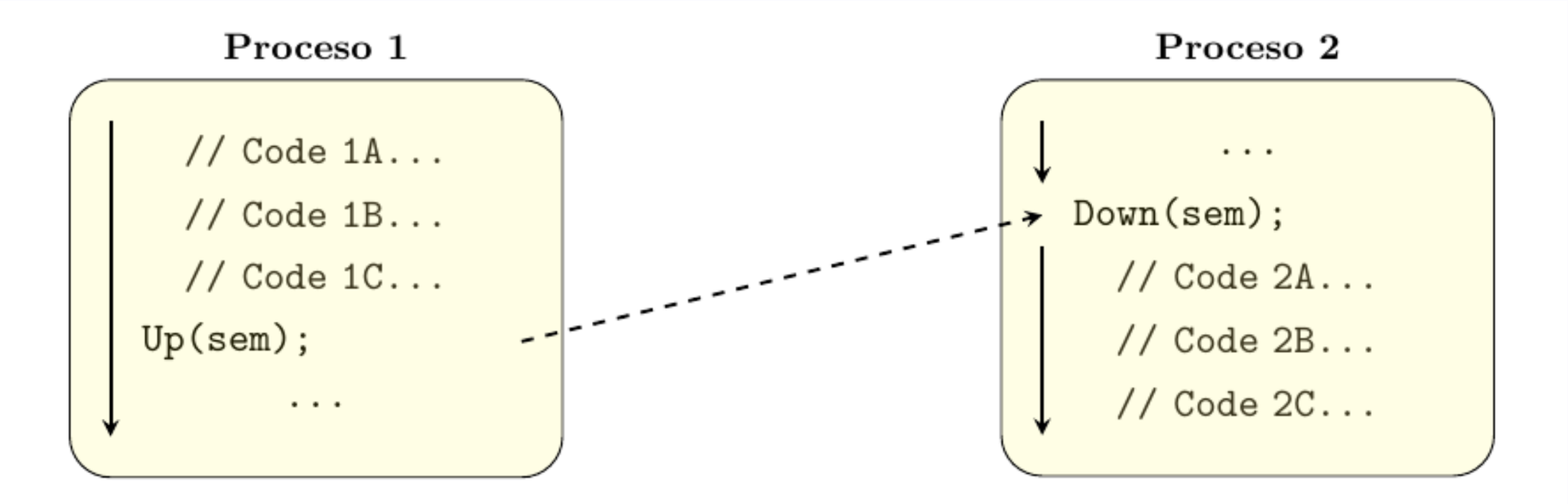


Figura 2: Semáforo inicializado a 0 que garantiza que el bloque «1» de Proceso 1 se ejecute antes que el bloque «2» de Proceso 2. Proceso 2 queda bloqueado en `Down(sem)` hasta que Proceso 1 ejecuta `Up(sem)`.

Semáforo para exclusión mutua

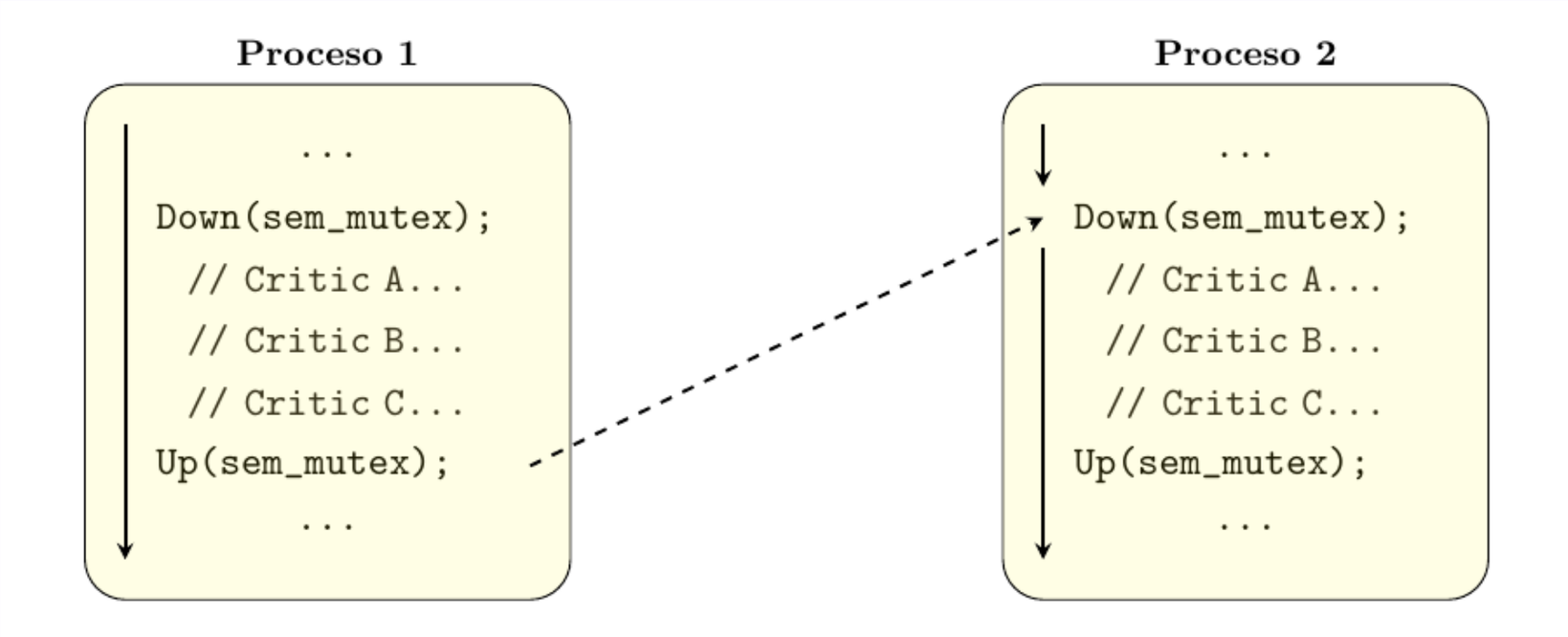


Figura 3: Semáforo inicializado a 1 para exclusión mutua. Solo un proceso puede estar en la región crítica al mismo tiempo. El Proceso 1 entra primero; el Proceso 2 queda bloqueado hasta que el Proceso 1 libera el semáforo.

Creación y eliminación de semáforos POSIX

Semáforos sin nombre

Deben estar en memoria accesible a todos los hilos/procesos que los usen.

Semáforos con nombre

Accesibles por cualquier proceso que conozca su nombre (`/nombre`). En Linux se crean en `/dev/shm`.

`sem_open`

Crea o abre un semáforo con nombre. Con `O_CREAT` se especifican permisos y valor inicial.

`sem_close`

Cierra el semáforo liberando recursos del proceso. El semáforo sigue accesible a otros.

`sem_unlink`

Elimina el nombre del semáforo. Los recursos se liberan cuando el contador de procesos llega a cero.

Operaciones con semáforos POSIX

Operaciones principales

- `sem_post`: operación **Up**
- `sem_wait`: operación **Down** (bloqueante)
- `sem_trywait`: Down no bloqueante
- `sem_timedwait`: Down con límite de tiempo
- `sem_getvalue`: consultar valor actual

Compilación

Las funciones están en `<semaphore.h>`. Es necesario enlazar con la biblioteca de hilos:

```
gcc programa.c -o programa -lpthread
```

Consultar `sem_overview` en el manual para un resumen completo.

Ejercicio 7: Creación de semáforos (`sem_create.c`)

```
#define SEM_NAME "/example_sem"
int main(void) {

    sem_t *sem = NULL;
    pid_t pid;

    sem = sem_open(SEM_NAME, O_CREAT|O_EXCL, S_IRUSR|S_IWUSR, 0);

    sem_print(sem); /* valor: 0 */

    sem_post(sem); sem_print(sem); /* valor: 1 */

    sem_post(sem); sem_print(sem); /* valor: 2 */

    sem_wait(sem); sem_print(sem); /* valor: 1 */

    pid = fork();

    if (pid == 0) {

        sem_wait(sem); /* región crítica hijo */

        sleep(5);

        sem_post(sem); sem_close(sem); exit(EXIT_SUCCESS);

    } else {

        sem_wait(sem); /* región crítica padre */

        sleep(5);

        sem_post(sem); sem_close(sem);

        sem_unlink(SEM_NAME); wait(NULL); exit(EXIT_SUCCESS);

    }

}
```

Aviso: versión reducida del código para las diapositivas. Consultar el enunciado

¿Podría modicarse el sitio de llamada a sem_unlink? En caso armativo, ¿cuál sería la primera posición en la que se sería correcto llamar a sem_unlink?

Ejercicio 8: Semáforos y señales (`sem_signal.c`)

```
void handler(int sig) { return; }
int main(void) {
    sem_t *sem = sem_open(SEM_NAME, O_CREAT|O_EXCL, S_IRUSR|S_IWUSR, 0);

    act.sa_handler = handler;

    sigaction(SIGINT, &act, NULL);

    printf("Starting wait (PID=%d)\n", getpid());

    sem_wait(sem);

    printf("Finishing wait\n");

    sem_unlink(SEM_NAME);
}
```

Aviso: versión reducida del código para las diapositivas. Consultar el enunciado

¿Qué sucede cuando se envía la señal SIGINT? ¿La llamada a Sem_wait se ejecuta con éxito?

¿Qué sucede si, en lugar de Usar un manejador vacío, se ignora la señal con SIG_IGN

¿Qué cambios habría que hacer en el programa anterior para garantizar que no termine salvo que consiga hacer Down el semáforo, lleguen o no señales?

Concurrencia: Ejercicio 9 — Procesos alternos

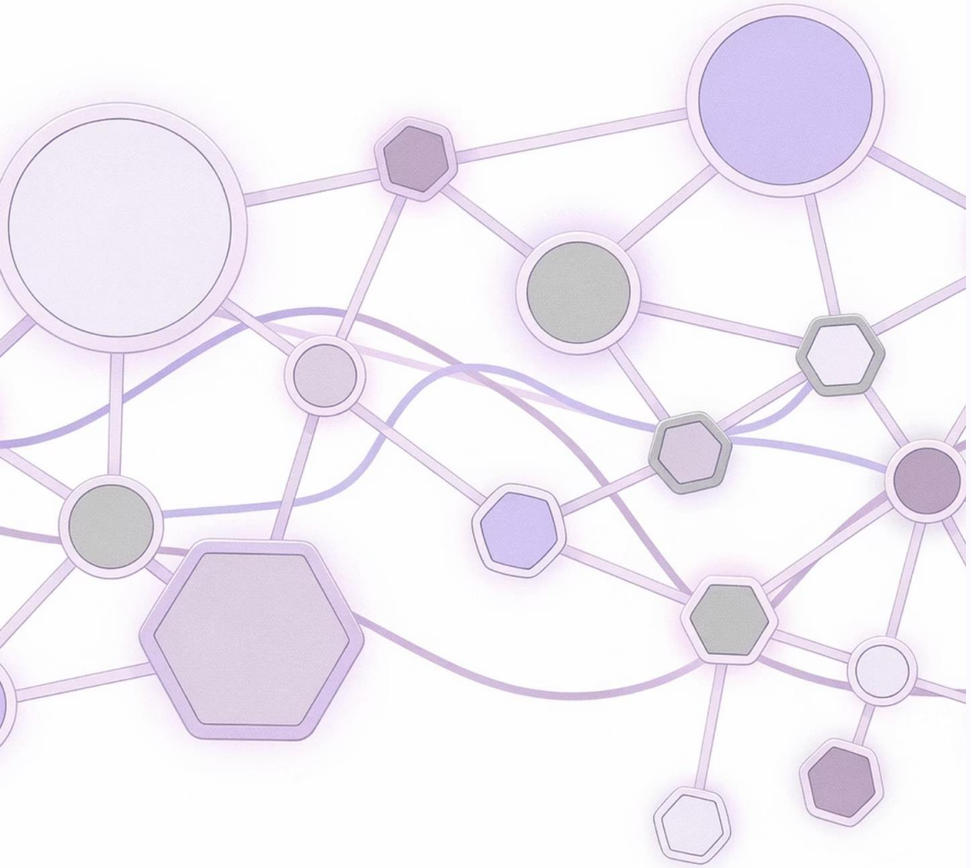
Rellenar los huecos A–F en `conc_alternate.c` para que la salida sea `1 2 3 4` en orden:

Proceso hijo (pid == 0)

```
/* A:  */
printf("1\n");
/* B:  */
printf("3\n");
/* C:  */
```

Proceso padre (pid > 0)

```
/* D:  */
printf("2\n");
/* E:  */
printf("4\n");
/* F: */
```



Ejercicio de codificación

Contexto: Red de mineros Blockchain

Continuando con la implementación de la práctica 1, se añade un **sistema de votación multiproceso** para validar soluciones. Cada ronda, un proceso se proclama «candidato» y el resto vota si acepta su propuesta. La sincronización se realiza mediante **señales y semáforos**.

3.5	3.5	2
Parte a)	Parte b)	Parte c) + d)
Múltiples procesos Minero.	Sistema de votaciones.	Protección zonas críticas y análisis.

Parte a) — Múltiples procesos Minero (3,5 ptos.)

Arranque: `./miner <N_SECS> <N_THREADS>`

→ Fichero de sistema protegido

Almacena los PID de los procesos Minero activos, protegido por un semáforo mutex.

→ Incorporación dinámica

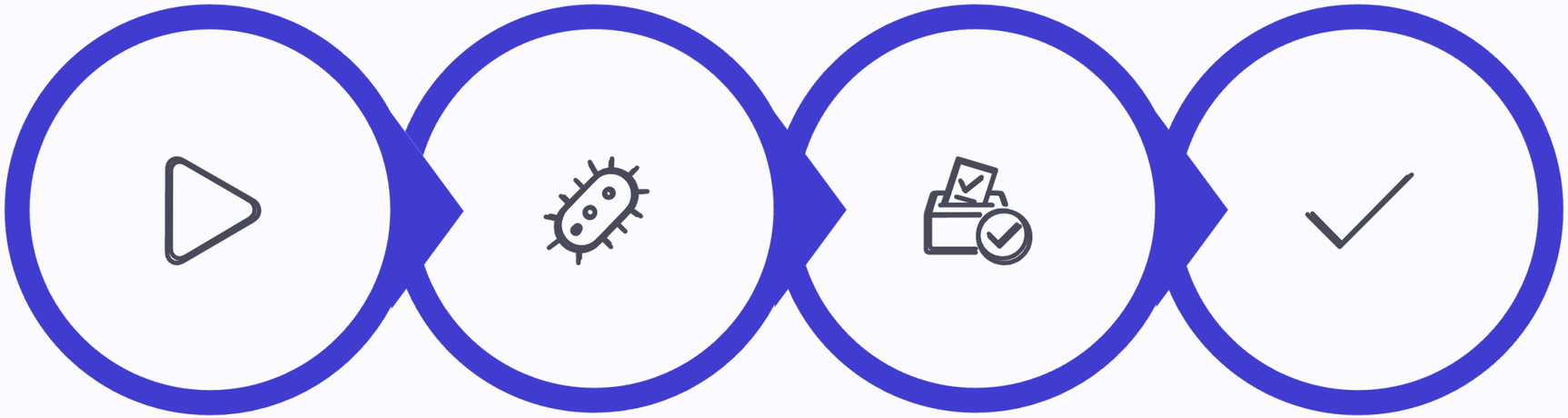
Cualquier minero lanzado se une apuntando su PID. Los mineros pueden unirse en cualquier momento desde la misma o distinta shell.

→ Salida y limpieza

Al terminar (`N_SECS`), el minero elimina su PID del fichero. El último minero destruye el fichero del sistema.

```
$ ./miner 5 8 & ./miner 10 1
Miner 19427 added to system
Miner 19428 added to system
Miner 19427 exited system
```

Parte b) — Sistema de votaciones (3,5 ptos.)



Arrancar Ganador Votación Conteo

El primer minero establece el objetivo inicial, envía **SIGUSR1** para arrancar la ronda. El ganador escribe la solución, envía **SIGUSR2** para votar. Si hay mayoría de votos válidos, el ganador recibe una moneda.

Flujo detallado del sistema de votación

01

Primer minero

Crea fichero de PIDs y fichero de objetivo (target). Envía SIGUSR1 al resto para arrancar la primera ronda.

02

Competición por ser Ganador

Todos compiten mediante `sem_trywait`. El primero en obtener solución se proclama Ganador y escribe en `Target.tgt`.

03

Votación

El Ganador envía SIGUSR2. Cada minero comprueba la solución y escribe **Y** o **N** en `Voting.vot` (protegido por mutex).

04

Recuento y nueva ronda

Si votos Y ≥ votos N, el Ganador recibe una moneda. Actualiza el objetivo y envía SIGUSR1 para la siguiente ronda.

Condiciones del sistema de votación

Mínimo de participantes

Se necesitan al menos 2 mineros para realizar una ronda. Con un solo proceso, se espera a que se incorporen más.

Incorporación en ronda

Un minero que se une espera la señal SIGUSR1 para no entrar en medio de una ronda y evitar inconsistencias.

Espera de votos

El Ganador hace esperas no activas cortas hasta que todos votan o se supera un número máximo de intentos, evitando bloqueos si un minero termina abruptamente.

Variación de participantes

Al inicio de cada ronda puede variar el número de procesos porque alguno haya agotado su tiempo de vida.

Ejemplo de ejecución

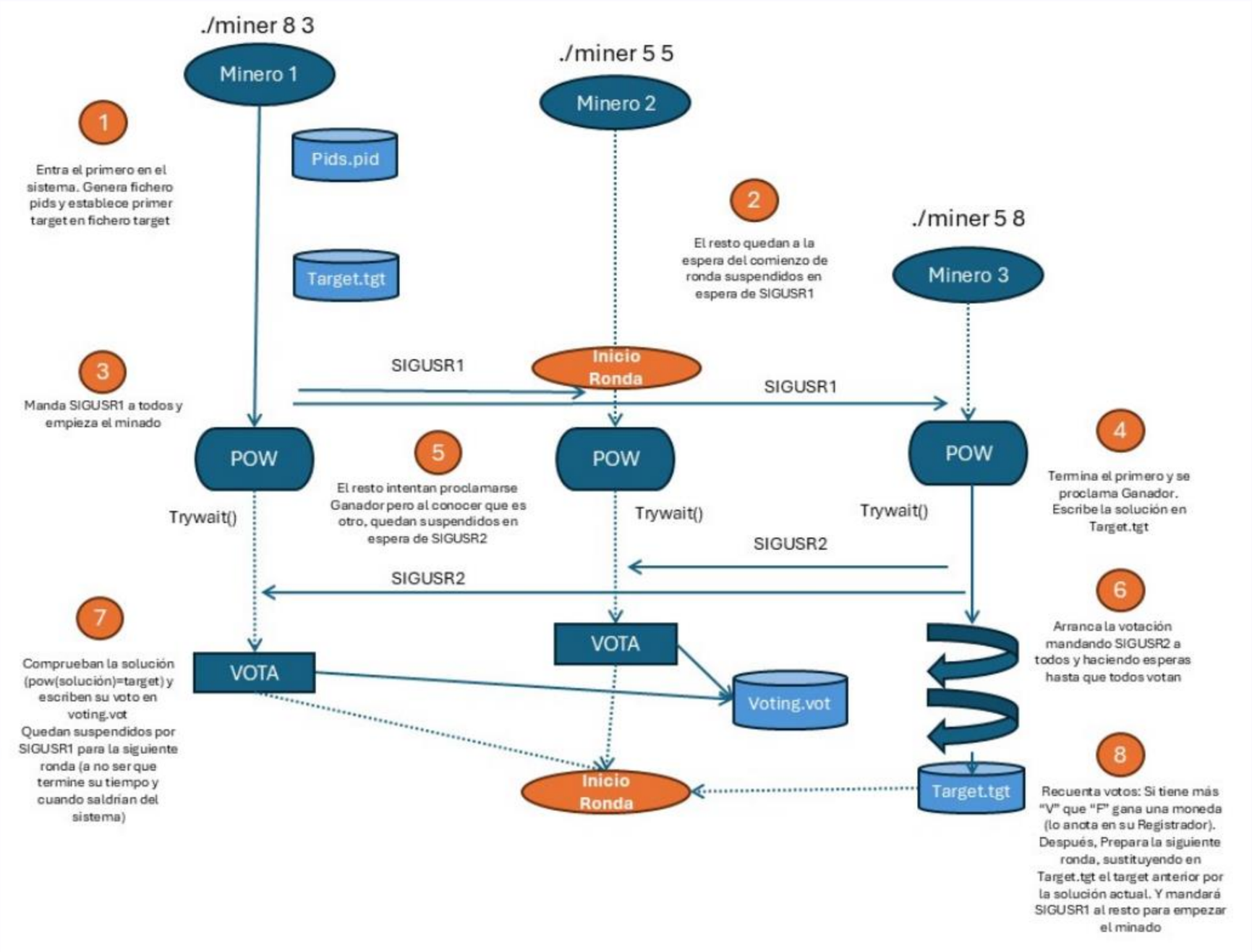
```
$ ./miner 3 8 & ./miner 5 8 & ./miner 10 8 & ./miner 10 1
Winner 19427 => [ N ] => Rejected
Winner 19426 => [ Y Y ] => Accepted
Winner 19427 => [ Y Y Y ] => Accepted
Winner 19425 => [ Y Y Y ] => Accepted
Winner 19426 => [ Y Y Y ] => Accepted
Winner 19426 => [ Y Y ] => Accepted
```

Fichero registrador del proceso 19426:

```
$ tail -6 19426.txt
Id:       6
Winner:   19426
Target:   44638907
Solution: 98780967 (validated)
Votes:    3/3
Wallets:  19426:3
```

El proceso 19426 ganó 3 rondas aceptadas, acumulando 3 monedas en su cartera.

Esquema global del sistema de votación



Flujo de eventos de una ronda

Minero 1 entra al sistema

Genera fichero de PIDs y establece primer target en `Target.tgt`.

Resto esperan SIGUSR1

Los demás mineros quedan suspendidos esperando el inicio de ronda.

Minero 1 envía SIGUSR1

Todos los mineros comienzan el minado (POW).

Minero 3 se proclama Ganador

Escribe la solución en `Target.tgt`. El resto quedan suspendidos esperando SIGUSR2.

Votación con SIGUSR2

Cada minero comprueba la solución y escribe su voto en `Voting.vot`.

Recuento y nueva ronda

El Ganador recuenta, anota monedas si procede, actualiza target y envía SIGUSR1.

Parte c) — Protección de zonas críticas (2 ptos.)

Para garantizar la robustez del sistema y que no se pierda ninguna señal:

- Bloquear señales con `sigprocmask`
- Esperas no activas con `sigsuspend`

Parte d) — Análisis de la ejecución

¿Problemas de concurrencia?

Solución adoptada

¿Solo con señales?

¿Quién gana más monedas?

Aspectos a tener en cuenta en la entrega

Documentación en PDF

Entregar la documentación en formato .pdf según la normativa de prácticas.

Control de errores

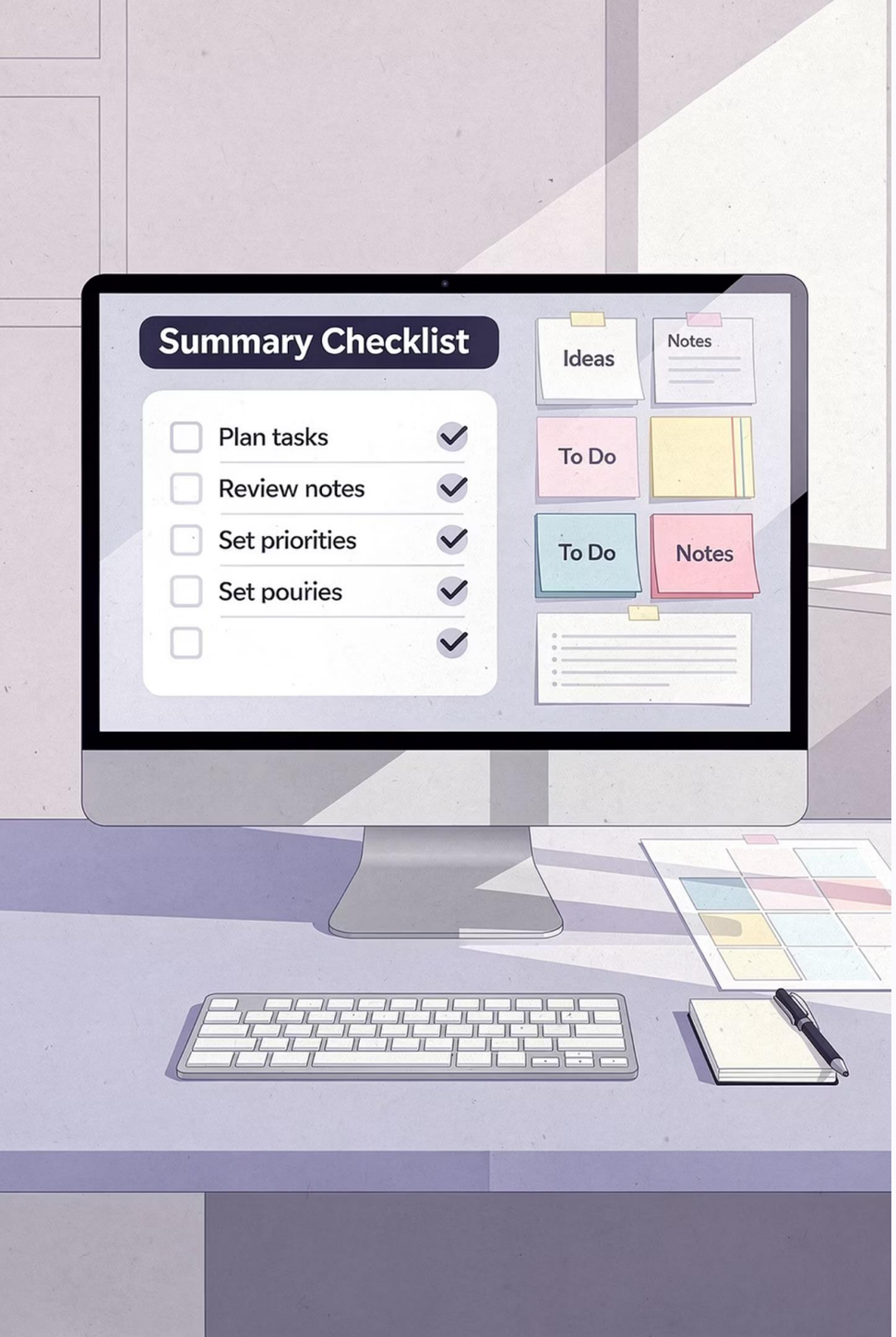
Comprobar el valor de retorno de todas las llamadas al sistema y gestionar los errores adecuadamente.

Liberación de semáforos

Garantizar que todos los semáforos se cierran (`sem_close`) y eliminan (`sem_unlink`) correctamente al finalizar.

Sin pérdida de señales

Garantizar que no se producen bloqueos por pérdida de señales usando `sigsuspend` y bloqueo previo con `sigprocmask`.



Resumen de la práctica

Señales POSIX

Envío con `kill`, captura con `sigaction`, bloqueo con `sigprocmask`, espera atómica con `sigsuspend`, temporización con `alarm`.

Semáforos POSIX

Creación con `sem_open`, operaciones Down/Up con `sem_wait`/`sem_post`, eliminación con `sem_unlink`.

Sistema de votación

Integración de señales y semáforos en un sistema multiproceso de minería Blockchain con votación distribuida.